UNIT 4 · TOPIC 4.2

Device Authentication Hardening

Assignment

Instructions: 23 practice questions + 4 coding assignment(s), 39 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

4.2.a · Passwords and password policies

1. According to NIST SP 800-63B, which practice is recommended when a user creates a new password? (1 pt)

- ○ A. Require at least one uppercase letter, one digit, and one symbol
- ○ B. Force the user to change the password every 90 days
- ○ C. Check the candidate against a list of known-compromised passwords and reject matches
- ○ D. Block copy-paste to prevent shoulder surfing

2. Which of the following are valid reasons an organization enforces an account lockout policy? (Select ALL that apply.) (2 pts)

- □ A. Prevents automated online guessing attacks
- □ B. Eliminates the need for password hashing
- □ C. Forces attackers to slow down their credential testing
- □ D. Makes brute-force infeasible in an interactive login scenario

(Select ALL that apply.)

3. An attacker obtains a copy of a site's password database. The passwords are stored with bcrypt. Why does this algorithm make cracking harder than MD5? (1 pt)

- ○ A. Bcrypt produces longer hashes, so there are more possible values to try
- ○ B. Bcrypt hashes are encrypted with a server key the attacker does not have
- ○ C. Bcrypt changes each user's hash every time they log in
- ○ D. Bcrypt is intentionally slow, making each guess computationally expensive

4. What is the primary purpose of a cryptographic salt in password storage? (1 pt)

- ○ A. To make the password longer before the user sees it
- ○ B. To encrypt the password so it can be decrypted later
- ○ C. To ensure two users with the same password have different stored hashes
- ○ D. To enforce complexity rules automatically

5. Name the two attack types that increasing password LENGTH most directly defends against. (1 pt)

4.2.b · Multi-factor authentication

6. A user logs in with a password, then enters a memorized answer to a security question. Is this true multi-factor authentication? (1 pt)

- ○ A. No, both credentials are 'something you know', the same factor category
- ○ B. Yes, two credentials were entered
- ○ C. Yes, because a security question is a possession factor
- ○ D. No, security questions are never allowed in authentication

7. Which factor category does a hardware security key (like a FIDO2/YubiKey device) belong to? (1 pt)

- ○ A. Something you know
- ○ B. Something you have (possession)
- ○ C. Something you are (inherence)
- ○ D. It is not a factor at all

8. Select ALL statements that are TRUE about SMS-based one-time codes used for MFA. (2 pts)

- □ A. They are vulnerable to SIM-swapping attacks
- □ B. They are immune to phishing because the code changes every time
- □ C. They can be intercepted or relayed by an adversary-in-the-middle phishing kit
- □ D. They still combine a knowledge factor (password) with a possession factor (phone)

(Select ALL that apply.)

9. An attacker who already has a victim's password sends them ten push-approval MFA prompts in a row until the victim taps Approve just to stop the notifications. What is this attack called? (1 pt)

- ○ A. SIM swapping
- ○ B. MFA fatigue (push bombing)
- ○ C. TOTP replay
- ○ D. Credential stuffing

10. Why is FIDO2/WebAuthn considered phishing-resistant compared to a typed authenticator-app code? (1 pt)

- ○ A. It cryptographically binds the response to the legitimate site's exact domain, so a fake login page cannot obtain a valid response
- ○ B. It never requires a second device
- ○ C. It does not require a password at all, ever
- ○ D. It uses a longer numeric code than TOTP

11. Name the attack in which an adversary tricks or bribes a mobile carrier into porting a victim's phone number to an attacker-controlled SIM card, in order to intercept SMS-based MFA codes. (1 pt)

4.2.c · Session security

12. An attacker sends a victim a link containing a session ID the attacker already knows. The victim logs in and the server keeps using that same ID. Which attack is this? (1 pt)

- ○ A. Session hijacking
- ○ B. SQL injection

○ C. Session fixation

○ D. Brute force

13. Which cookie attribute prevents JavaScript (including injected XSS scripts) from reading the session cookie? (1 pt)

○ A. HttpOnly

○ B. Secure

○ C. SameSite

○ D. Max-Age

14. Select ALL statements that are TRUE about session timeouts. (2 pts)

□ A. An idle timeout ends a session after a period of no activity

□ B. An absolute timeout ends a session after a fixed total duration regardless of activity

□ C. A session with no timeout configured poses no additional risk

□ D. Timeouts limit how long a stolen or fixated token remains useful

(Select ALL that apply.)

15. A user clicks 'Log Out' and their browser cookie is deleted, but the server never marks the session record as invalid. What is the consequence? (1 pt)

○ A. No consequence; deleting the cookie is sufficient

○ B. The user's password is exposed

○ C. A token captured earlier by an attacker remains valid and usable

○ D. The session is automatically regenerated

16. Name the defensive technique of issuing a brand-new session ID at login (and at privilege change) that specifically defeats session fixation. (1 pt)

17. A session cookie is sent only over HTTPS due to one cookie flag, but is still readable by page JavaScript because a different flag is missing. Which flag is MISSING? (1 pt)

○ A. HttpOnly

○ B. Secure

○ C. Domain

○ D. Path

4.2.d · Least privilege and access control models

18. What does the principle of least privilege (PoLP) require? (1 pt)

○ A. A subject should be granted only the minimum access needed for its job function

○ B. Every user should have the same access, so no one is treated unfairly

○ C. Access should be granted generously at first, then reduced if problems appear

○ D. Only administrators need access controls; regular users are low risk

19. A file's owner directly decides who else may read or write it, without any central authority involved. Which access control model is this? (1 pt)

○ A. MAC

○ B. RBAC

○ C. ABAC

○ D. DAC

20. A system grants access dynamically based on the user's department, the sensitivity label of the resource, and whether the request happens during business hours. Which model is this? (1 pt)

○ A. ABAC

○ B. MAC

○ C. RBAC

○ D. DAC

21. Select ALL statements that are TRUE about over-privileged accounts. (2 pts)

□ A. They increase the blast radius if the account is compromised

□ B. They eliminate insider-threat risk because trusted employees hold them

□ C. Malware executed under an over-privileged session inherits that account's broad access

□ D. They make ransomware propagation to additional systems more likely

(Select ALL that apply.)

22. Name the term for the silent accumulation of unnecessary access rights over time as an employee's role changes. (1 pt)

23. Which statement correctly distinguishes RBAC from ABAC? (1 pt)

○ A. RBAC lets owners share files, while ABAC uses security labels

○ B. RBAC assigns static permissions via roles; ABAC evaluates dynamic attributes at request time

○ C. RBAC is used for networks, while ABAC is used for physical doors

○ D. RBAC checks the user only, while ABAC checks the user's password strength

Coding Assignments (Python 3)

4.2.a · Passwords and password policies

3 pts · auto-graded on the site

Coding problem 4.2.a: Passwords and password policies

Skill: Mitigate Risk · **Maps to:** 4.2.A · **Language:** Python 3 (pytest)

Problem

A system administrator has defined a password policy (minimum length, complexity requirements, a banned/breached-password list, and a history of previously used passwords). Build the checker that evaluates a candidate password against that policy and reports every rule it violates, so the login form can show the user exactly what to fix.

Function contract

Implement `validate_password(password: str, policy: dict) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `password`: a string, the candidate password.

- `policy`: a dict. Example: ``python policy = { "min_length": 12, "require_upper": True, "require_lower": True, "require_digit": True, "require_symbol": True, "banned_list": ["password123", "letmein", "qwerty123456"], "history": ["OldPass2023!", "OldPass2024!"], } Any key may be omitted; a missing key means that rule is not enforced (see interface.md` for exact defaults).

Output

A `list[str]` of violated rule names, in the fixed order defined in `interface.md`: `too_short`, `no_upper`, `no_lower`, `no_digit`, `no_symbol`, `in_banned_list`, `reused_password`. An empty list `[]` means the password is fully compliant.

Examples

```
validate_password("abc", {"min_length": 12, "require_upper": True, "require_digit": True})
    -> ["too_short", "no_upper", "no_digit"]

validate_password("Password123", {"banned_list": ["password123"]})
    -> ["in_banned_list"]

validate_password(
    "Tr0ub4dor&3xtra!",
    {"min_length": 12, "require_upper": True, "require_lower": True,
     "require_digit": True, "require_symbol": True,
     "banned_list": ["password123"], "history": ["OldPass2023!"]},
) -> []
```

Constraints

- Deterministic, offline, standard library only.

- Missing policy keys are treated as "not required" (see `interface.md` for exact defaults).

- Violated rule names must be returned in the fixed order given above, even when several rules are violated at once.

- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 4.2.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""


def validate_password(password: str, policy: dict) -> list:
    """Validate a candidate password against a policy dict.

    Args:
        password: The candidate password string.
        policy: A dict describing the policy. Recognized keys (all optional):
            min_length (int, default 0)          – minimum required length.
            require_upper (bool, default False)  – require an uppercase letter.
            require_lower (bool, default False)  – require a lowercase letter.
            require_digit (bool, default False)  – require a digit.
            require_symbol (bool, default False) – require a non-alphanumeric symbol.
```

```
        banned_list (list[str], default [])  — forbidden/breached passwords,
            matched case-insensitively.
        history (list[str], default [])      — previously used passwords, matched
            case-sensitively (exact match) to prevent reuse.

    Returns:
        A list of violated rule-name strings, in this fixed order:
        ["too_short", "no_upper", "no_lower", "no_digit", "no_symbol",
         "in_banned_list", "reused_password"]. Only the names of violated rules
        are included. Returns [] if the password is fully compliant.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement validate_password() — see prompt.md")
```

4.2.b · Multi-factor authentication

3 pts · auto-graded on the site

Coding problem 4.2.b: Classify a login attempt's MFA strength

Skill: Mitigate Risk · **Maps to:** 4.2.A, 4.2.B · **Language:** Python 3 (pytest)

Problem

A security analyst receives, for each login attempt, the list of authentication factors that were presented (e.g. `["password", "sms_code"]`). Build the classifier that determines whether the attempt satisfies true multi-factor authentication, and if so, whether it relies on a phishing-resistant possession factor or a phishable one.

Categories of result:

- `"single_factor"`: the presented factors, once mapped to their factor category (knowledge / possession / inherence), cover fewer than two distinct categories. This includes presenting only one factor, presenting several factors from the *same* category (e.g. two passwords), an empty list, or a list of only unrecognized factor strings.
- `"mfa_phishing_resistant"`: two or more distinct factor categories are present, and at least one of the presented factors is a phishing-resistant possession factor (`hardware_key` or `fido2`).
- `"mfa_phishable"`: two or more distinct factor categories are present, but no phishing-resistant possession factor was used (e.g. it relies on `sms_code`, `totp_code`, `authenticator_app`, `push_approved`, or `backup_code` for the possession factor).

The exact factor-to-category mapping and full rule set are defined in `interface.md`, implementations must follow it precisely.

Function contract

Implement `solve(factors: list[str]) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of strings, each a recognized or unrecognized factor name. May be empty.

Output

One of: `"single_factor"`, `"mfa_phishable"`, `"mfa_phishing_resistant"`.

Examples

```
["password"]                        -> "single_factor"
["password", "password"]            -> "single_factor"
["password", "security_question"]   -> "single_factor"
["password", "sms_code"]            -> "mfa_phishable"
["password", "push_approved"]       -> "mfa_phishable"
```

```
["password", "hardware_key"]          -> "mfa_phishing_resistant"
["password", "sms_code", "hardware_key"] -> "mfa_phishing_resistant"
["biometric", "password"]             -> "mfa_phishable"
[]                                     -> "single_factor"
```

Constraints

- Deterministic, offline, standard library only.
- Follow the exact factor -> category mapping in `interface.md`.
- Unrecognized factor strings must not raise an error; they are simply ignored.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 4.2.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import List


def solve(factors: List[str]) -> str:
    """Classify a login attempt's MFA strength from its presented factors.

    Args:
        factors: list of authentication-factor strings presented during one login
            attempt (e.g. ["password", "hardware_key"]). May be empty.

    Returns:
        One of "single_factor", "mfa_phishable", "mfa_phishing_resistant".
        See interface.md for the exact factor -> category mapping and rules.
    """
    # TODO: implement per prompt.md / interface.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

4.2.c · Session security

3 pts · auto-graded on the site

Coding problem 4.2.c: Classify session-security incidents

Skill: Mitigate Risk · **Maps to:** 4.2.B · **Language:** Python 3 (pytest)

Problem

A security analyst receives short incident-report strings describing observed session behavior. Build the classifier that maps each description to the correct session-security category.

Categories:

- `"hijacking"`: an already-authenticated session token is stolen (XSS read, network sniffing, malware) and reused by an attacker
- `"fixation"`: an attacker supplies a session ID to the victim before login, then reuses that same ID after the victim authenticates
- `"insecure_cookie"`: the session cookie is missing a secure attribute (HttpOnly, Secure, or SameSite) that would have prevented exposure or misuse

- `"timeout_issue"`: a session remains valid far longer than it should because of a missing or misconfigured idle/absolute timeout
- `"secure"`: the report describes a session that is properly protected (regeneration, secure flags, timeouts all present) with no vulnerability
- `"unknown"`: nothing in the report matches any category

A report may match exactly one category. If no category matches, return `"unknown"`.

Function contract

Implement `solve(report: str) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the single category label string. Matching is case-insensitive substring detection using the indicator sets defined in `interface.md`.

Input

A single string describing observed session behavior on a device, browser, or server.

Output

One of: `"hijacking"`, `"fixation"`, `"insecure_cookie"`, `"timeout_issue"`, `"secure"`, `"unknown"`.

Examples

```
"Session token was sniffed over unencrypted HTTP and replayed by an attacker"        -> "hijacking"
"Attacker sent a link with a pre-set session id; victim logged in and id unchanged"  -> "fixation"
"Session cookie has no HttpOnly flag configured; automated scan flagged the issue"    -> "insecure_cookie"
"Logged-in kiosk left unattended for hours with no idle timeout configured"          -> "timeout_issue"
"Session id regenerated at login; cookie set Secure, HttpOnly, SameSite=Strict"      -> "secure"
"User changed their display name in account settings"                                -> "unknown"
```

Constraints

- Deterministic, offline, standard library only.
- Detection by case-insensitive substring match against indicator sets in `interface.md`.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 4.2.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""


def solve(report: str) -> str:
    """Classify a session-security incident report by its behavioral signature.

    Args:
        report: A string containing the incident report description.

    Returns:
        The category string: "hijacking", "fixation", "insecure_cookie",
        "timeout_issue", "secure", or "unknown".
    """
```

```
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

4.2.d · Least privilege and access control models

3 pts · auto-graded on the site

Coding problem 4.2.d: Access-request auditor

Skill: Mitigate Risk · **Maps to:** 4.2.B, 4.2.C · **Language:** Python 3 (pytest)

Problem

A security team reviews access requests before granting them. Each request either has a business justification attached or it does not; the requester's access may or may not have been reviewed recently; and the requested access may be scoped to a temporary time window or be standing (permanent). Build the auditor function that applies the least-privilege triage rules to a single request and returns the decision.

Function contract

Implement `def solve(request: dict) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A dict with keys `current_role` (str), `requested_access` (str), `justification_present` (bool), `time_bound` (bool or None), `last_reviewed_days_ago` (int). Optional keys default as documented in `interface.md` (missing `justification_present` defaults to `False`; missing `time_bound` defaults to `False`; missing `last_reviewed_days_ago` defaults to `0`).

Output

One of the strings: `"approve"`, `"deny_no_justification"`, `"flag_privilege_creep"`, `"approve_time_bound"`.

Decision rule (apply in this exact order, first match wins)

1. No justification (`justification_present` is `False`) → `"deny_no_justification"`. 2. Else, stale review (`last_reviewed_days_ago > 365`) → `"flag_privilege_creep"`. 3. Else, time-bound request (`time_bound` is `True`) → `"approve_time_bound"`. 4. Else → `"approve"`.

Rule 1 is checked before rule 2, a request with no justification is denied even if it is also years overdue for review or time-bound. The staleness threshold is 365 days, strictly greater than (365 itself is still considered current).

Examples

```
{"current_role": "Marketing Intern", "requested_access": "prod-db-write",
 "justification_present": False, "time_bound": False, "last_reviewed_days_ago": 10}
    -> "deny_no_justification"

{"current_role": "HR Analyst", "requested_access": "payroll-admin",
 "justification_present": True, "time_bound": False, "last_reviewed_days_ago": 800}
    -> "flag_privilege_creep"

{"current_role": "On-call Engineer", "requested_access": "prod-ssh",
 "justification_present": True, "time_bound": True, "last_reviewed_days_ago": 5}
    -> "approve_time_bound"

{"current_role": "Analyst", "requested_access": "reporting-read",
 "justification_present": True, "time_bound": False, "last_reviewed_days_ago": 30}
    -> "approve"
```

Constraints

- Deterministic, offline, standard library only.
- Missing optional keys must not raise, use the documented defaults.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 4.2.d. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""
from typing import Any


def solve(request: dict) -> str:
    """Audit a single access request per the least-privilege priority rules.

    Args:
        request: dict with keys current_role (str), requested_access (str),
            justification_present (bool), time_bound (bool or None),
            last_reviewed_days_ago (int). Optional keys default per interface.md.

    Returns:
        One of: "approve", "deny_no_justification", "flag_privilege_creep",
        "approve_time_bound".
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```